

Vermont Health Platform — User Management Guide

This guide is written for platform administrators who have access to the Supabase SQL Editor but are not developers. Every operation includes exact SQL you can copy and run directly.

Table of Contents

- 1. [Understanding the Role System](#)
- 2. [Creating User Accounts](#)
- 3. [Granting and Revoking Roles](#)
- 4. [Common User Management Tasks](#)
- 5. [Handling Support Requests](#)
- 6. [Subscription and Billing](#)
- 7. [Monitoring Users](#)

1. Understanding the Role System

The Six Roles

Roles are stored in the `public.user_roles` table. Every user must have at least the `free` role, which is assigned automatically on signup.

Role	What It Unlocks	Notes
<code>free</code>	Public content only — articles, academy previews, state map, about pages	Assigned automatically to every new user
<code>subscriber</code>	<code>/dashboard</code> (Rural Health Transformation data), <code>/chat</code> (AI Analyst), <code>/hti-dashboard</code> (HTI Dashboard)	Requires paid subscription or manual grant
<code>student</code>	Academy coursework and learning tracks	For enrolled students
<code>professional</code>	Professional-tier features (reserved for future use)	
<code>advisory</code>	<code>/advisory-hub</code> — advisory services portal and private reports	For clients of the advisory practice
<code>admin</code>	Everything — bypasses all role checks, can manage all data	Grant sparingly

How Multiple Roles Work

A user can hold several roles simultaneously. For example, a user could be both `subscriber` and `student` at the same time. Each role is a separate row in `user_roles`.

```
user_id: abc-123
└─ role: free      (granted at signup)
└─ role: subscriber (granted when they paid)
└─ role: student   (granted when enrolled)
```

How the Middleware Checks Roles

Every page request passes through `frontend/middleware.ts`. When a user tries to visit a protected route, the middleware:

- 1. Checks whether the user is logged in. If not, redirects to `/login?from=[page]`.
- 2. Queries `public.user_roles` for all roles the user holds.
- 3. Checks whether any of those roles meets the requirement for that route.

4. If the check fails, redirects to `/upgrade?from=[page]`.

The middleware uses a role hierarchy for comparisons:

```
free → subscriber → student → professional → advisory → admin
```

Roles with a higher position in the hierarchy satisfy requirements for lower positions. **admin** satisfies any role check. This means an admin can access `/dashboard` even without an explicit `subscriber` row.

Protected Routes Reference

Route	Requires
<code>/dashboard</code>	<code>subscriber</code> (or higher)
<code>/chat</code>	<code>subscriber</code> (or higher)
<code>/hti-dashboard</code>	<code>subscriber</code> (or higher)
<code>/advisory-hub</code>	<code>advisory</code> (or higher)
<code>/admin</code>	<code>admin</code> only
<code>/account</code>	Any authenticated user
<code>/onboarding</code>	Any authenticated user

2. Creating User Accounts

Self-Service Signup

Users sign up themselves at `/signup`. They enter their email and password. The moment the account is created in Supabase Auth, a database trigger (`handle_new_user`) automatically:

1. Creates a row in `public.profiles` with their email and name.
2. Creates a row in `public.user_roles` with role = `free`.
3. Creates a row in `public.subscriptions` with plan = `free`, status = `active`.

No admin action is needed for standard signup.

Creating an Account Manually in Supabase

If you need to create an account for someone (e.g., a comp account, an internal tester):

1. Go to your Supabase project dashboard.
2. Navigate to **Authentication** → **Users**.
3. Click **Add user** → **Create new user**.
4. Enter the user's email and a temporary password.
5. Check **Auto Confirm User** so they can log in immediately.
6. Click **Create User**.

The `handle_new_user` trigger fires automatically — you do not need to manually insert into `profiles` or `user_roles`.

Then send the user their temporary password and ask them to change it at `/account`.

Verifying That Setup Ran Correctly

After creating an account (by any method), confirm the trigger ran:

```
-- Replace the email address with the actual user's email
SELECT
```

```
p.email,  
p.full_name,  
p.created_at AS profile_created,  
ur.role,  
ur.granted_at,  
s.plan,  
s.status AS subscription_status  
FROM public.profiles p  
JOIN public.user_roles ur ON ur.user_id = p.id  
JOIN public.subscriptions s ON s.user_id = p.id  
WHERE p.email = 'user@example.com';
```

Expected result: at least one row with `role = free` and `plan = free`.

3. Granting and Revoking Roles

Finding a User's UUID

Nearly every operation requires the user's UUID (not their email). Get it with:

```
SELECT id, email, created_at  
FROM auth.users  
WHERE email = 'user@example.com';
```

Copy the `id` value — it looks like `a1b2c3d4-e5f6-7890-abcd-ef1234567890`.

Granting a Role

Use `INSERT ... ON CONFLICT DO NOTHING` so re-running is safe:

```
-- Grant subscriber role  
INSERT INTO public.user_roles (user_id, role)  
VALUES ('PASTE-USER-UUID-HERE', 'subscriber')  
ON CONFLICT (user_id, role) DO NOTHING;  
  
-- Grant student role  
INSERT INTO public.user_roles (user_id, role)  
VALUES ('PASTE-USER-UUID-HERE', 'student')  
ON CONFLICT (user_id, role) DO NOTHING;  
  
-- Grant advisory role  
INSERT INTO public.user_roles (user_id, role)  
VALUES ('PASTE-USER-UUID-HERE', 'advisory')  
ON CONFLICT (user_id, role) DO NOTHING;  
  
-- Grant admin role  
INSERT INTO public.user_roles (user_id, role)  
VALUES ('PASTE-USER-UUID-HERE', 'admin')  
ON CONFLICT (user_id, role) DO NOTHING;
```

Granting a Role With an Expiry Date

You can set roles to expire automatically (e.g., for trial access or a fixed-term subscription):

```
-- Grant subscriber access that expires in 30 days  
INSERT INTO public.user_roles (user_id, role, expires_at)  
VALUES ('PASTE-USER-UUID-HERE', 'subscriber', NOW() + INTERVAL '30 days')  
ON CONFLICT (user_id, role) DO UPDATE SET expires_at = NOW() + INTERVAL '30  
days';
```

Note: The middleware does not currently enforce `expires_at` automatically — expiry is stored for record-keeping and future automation. To cut off access, delete the row (see Revoking below).

Revoking a Role

```
-- Revoke a specific role from a user
DELETE FROM public.user_roles
WHERE user_id = 'PASTE-USER-UUID-HERE'
  AND role = 'subscriber';
```

This does not delete the user's account — only removes that one role. Their `free` role remains unless you also delete that.

Viewing All Roles a User Has

```
SELECT role, granted_at, expires_at, granted_by
FROM public.user_roles
WHERE user_id = 'PASTE-USER-UUID-HERE'
ORDER BY granted_at;
```

Viewing All Users With a Given Role

```
-- See all subscribers
SELECT p.email, p.full_name, ur.granted_at, ur.expires_at
FROM public.user_roles ur
JOIN public.profiles p ON p.id = ur.user_id
WHERE ur.role = 'subscriber'
ORDER BY ur.granted_at DESC;

-- Change 'subscriber' to any role: 'free', 'student', 'professional',
'advisory', 'admin'
```

4. Common User Management Tasks

Find a User's UUID by Email

```
SELECT id AS uuid, email, created_at
FROM auth.users
WHERE email = 'user@example.com';
```

Make Someone an Admin

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'admin')
ON CONFLICT (user_id, role) DO NOTHING;
```

Admins bypass all role checks and can see all data in the admin panel. Grant this only to trusted team members.

Upgrade Someone to Subscriber (After Manual Payment or Comp)

Step 1 — grant the role:

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;
```

Step 2 — update their subscription record to reflect the paid plan:

```
UPDATE public.subscriptions
SET plan = 'pro', status = 'active', updated_at = NOW()
WHERE user_id = 'PASTE-USER-UUID-HERE';
```

Give Someone Advisory Access

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'advisory')
ON CONFLICT (user_id, role) DO NOTHING;
```

If they are also a paying subscriber, grant **subscriber** separately:

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;
```

Remove a Role

```
DELETE FROM public.user_roles
WHERE user_id = 'PASTE-USER-UUID-HERE'
AND role = 'advisory';
```

List All Subscribers

```
SELECT
  p.email,
  p.full_name,
  p.org_name,
  ur.granted_at,
  ur.expires_at,
  s.plan,
  s.status,
  s.current_period_end
FROM public.user_roles ur
JOIN public.profiles p ON p.id = ur.user_id
LEFT JOIN public.subscriptions s ON s.user_id = ur.user_id
WHERE ur.role = 'subscriber'
ORDER BY ur.granted_at DESC;
```

See When a Role Was Granted

```
SELECT
  p.email,
  ur.role,
  ur.granted_at,
```

```
    ur.expires_at,
    ur.granted_by
FROM public.user_roles ur
JOIN public.profiles p ON p.id = ur.user_id
WHERE ur.user_id = 'PASTE-USER-UUID-HERE'
ORDER BY ur.granted_at;
```

See a Full User Summary

```
SELECT
  p.id,
  p.email,
  p.full_name,
  p.org_name,
  p.created_at AS account_created,
  s.plan,
  s.status AS sub_status,
  s.current_period_end,
  s.cancel_at_period_end,
  ARRAY_AGG(ur.role ORDER BY ur.granted_at) AS roles
FROM public.profiles p
LEFT JOIN public.subscriptions s ON s.user_id = p.id
LEFT JOIN public.user_roles ur ON ur.user_id = p.id
WHERE p.email = 'user@example.com'
GROUP BY p.id, p.email, p.full_name, p.org_name, p.created_at, s.plan,
s.status, s.current_period_end, s.cancel_at_period_end;
```

5. Handling Support Requests

User Says They Paid But Can't Access the Dashboard

- 1. Find their UUID:

```
SELECT id FROM auth.users WHERE email = 'user@example.com';
```

- 2. Check their current roles and subscription:

```
SELECT ur.role, ur.granted_at, s.plan, s.status, s.stripe_subscription_id
FROM public.user_roles ur
JOIN public.subscriptions s ON s.user_id = ur.user_id
WHERE ur.user_id = 'PASTE-USER-UUID-HERE';
```

- 3. If **subscriber** role is missing but payment was confirmed, grant it manually:

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;

UPDATE public.subscriptions
SET plan = 'pro', status = 'active', updated_at = NOW()
WHERE user_id = 'PASTE-USER-UUID-HERE';
```

- 4. Ask the user to refresh the page (or log out and back in). Role changes take effect on the next page load — no cache to clear.

User Locked Out of Account

If a user forgot their password, they can use the **Forgot Password** flow on the login page. This sends a password reset email through Supabase Auth.

If the reset email is not arriving or you need to force a reset as an admin:

1. Go to Supabase → **Authentication** → **Users**.
2. Find the user by email.
3. Click on their row, then click **Send password reset email**.

If their account was suspended or they are getting an "email not confirmed" error:

1. Go to Supabase → **Authentication** → **Users**.
2. Find the user and click their row.
3. Click **Confirm email** if it shows as unconfirmed.

User Wants to Cancel — How to Revoke Roles

Remove the **subscriber** role and downgrade their subscription:

```
-- Step 1: Remove subscriber role
DELETE FROM public.user_roles
WHERE user_id = 'PASTE-USER-UUID-HERE'
  AND role = 'subscriber';

-- Step 2: Update subscription record
UPDATE public.subscriptions
SET plan = 'free', status = 'canceled', updated_at = NOW()
WHERE user_id = 'PASTE-USER-UUID-HERE';
```

If they also have other premium roles (e.g., **advisory**) that should be removed:

```
DELETE FROM public.user_roles
WHERE user_id = 'PASTE-USER-UUID-HERE'
  AND role IN ('subscriber', 'advisory', 'student', 'professional');
-- This leaves their 'free' role intact so their account still works
```

Also cancel their Stripe subscription through the Stripe Dashboard if billing is active, to stop future charges.

User Account Cleanup / Deletion

Before deleting, document what the user had (for audit purposes):

```
-- Run this first and save the output
SELECT p.email, p.full_name, p.created_at, ur.role, ur.granted_at,
s.stripe_subscription_id
FROM public.profiles p
LEFT JOIN public.user_roles ur ON ur.user_id = p.id
LEFT JOIN public.subscriptions s ON s.user_id = p.id
WHERE p.email = 'user@example.com';
```

To delete the account:

1. Cancel their Stripe subscription in the Stripe Dashboard first (if applicable).
2. Go to Supabase → **Authentication** → **Users**.
3. Find the user, click their row, and click **Delete user**.

Because all tables use **ON DELETE CASCADE**, deleting from **auth.users** automatically removes:

- public.profiles
- public.user_roles (all roles)
- public.subscriptions
- public.stripe_customers
- public.course_enrollments
- public.module_progress
- public.certifications
- public.advisory_clients

You do not need to manually clean up those tables.

6. Subscription and Billing

How Stripe Webhooks Automatically Update Roles

When a user completes checkout through Stripe:

1. Stripe sends a webhook event to the platform.
2. The webhook handler (running server-side) receives the event and logs it to public.stripe_events for idempotency (preventing duplicate processing).
3. The handler updates public.subscriptions with the new plan, status, and period dates.
4. The handler inserts the subscriber role into public.user_roles.

This is the normal path. The user should gain access within seconds of completing payment.

What to Do When Automation Fails

Webhook failures can happen due to network issues, server restarts, or Stripe delivery retries. Signs of failure:

- User completed payment (has a receipt from Stripe) but still sees the /upgrade page.
- public.stripe_events does not contain the expected event for that user.
- public.subscriptions.status is not active.

Check the Stripe Dashboard → Developers → Webhooks → look for failed deliveries. You can resend a failed event from there, which will re-trigger the handler.

If you need to fix it immediately (manual override):

```
-- 1. Find the user
SELECT id FROM auth.users WHERE email = 'user@example.com';

-- 2. Grant subscriber role
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;

-- 3. Update subscription record with Stripe details
-- Get stripe_customer_id and stripe_subscription_id from Stripe Dashboard
UPDATE public.subscriptions
SET
  plan = 'pro',
  status = 'active',
  stripe_customer_id = 'cus_XXXXXXXXXX',
  stripe_subscription_id = 'sub_XXXXXXXXXX',
  current_period_start = NOW(),
  current_period_end = NOW() + INTERVAL '1 month',
  updated_at = NOW()
WHERE user_id = 'PASTE-USER-UUID-HERE';
```

Checking for Subscriptions Expiring Soon


```
SELECT
  p.email,
  p.full_name,
  s.plan,
  s.status,
  s.current_period_end,
  s.cancel_at_period_end,
  s.stripe_subscription_id
FROM public.subscriptions s
JOIN public.profiles p ON p.id = s.user_id
WHERE
  s.status = 'active'
  AND s.current_period_end < NOW() + INTERVAL '7 days'
  AND s.current_period_end > NOW()
ORDER BY s.current_period_end;
```

Checking for Past-Due Subscriptions

```
SELECT
  p.email,
  p.full_name,
  s.plan,
  s.status,
  s.current_period_end,
  s.stripe_subscription_id
FROM public.subscriptions s
JOIN public.profiles p ON p.id = s.user_id
WHERE s.status = 'past_due'
ORDER BY s.current_period_end;
```

Users with `past_due` status may have lost access if the webhook handler revoked their `subscriber` role. Confirm in `public.user_roles` and act based on your collections policy.

7. Monitoring Users

Total Users and Role Distribution

```
SELECT
  ur.role,
  COUNT(*) AS user_count
FROM public.user_roles ur
GROUP BY ur.role
ORDER BY
  CASE ur.role
    WHEN 'admin' THEN 1
    WHEN 'advisory' THEN 2
    WHEN 'professional' THEN 3
    WHEN 'student' THEN 4
    WHEN 'subscriber' THEN 5
    WHEN 'free' THEN 6
  END;
```

New Signups by Day (Last 30 Days)

```
SELECT
  DATE_TRUNC('day', created_at) AS signup_date,
  COUNT(*) AS new_users
```

```
FROM auth.users
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY 1
ORDER BY 1 DESC;
```

New Signups by Week (Last 12 Weeks)

```
SELECT
  DATE_TRUNC('week', created_at) AS week_start,
  COUNT(*) AS new_users
FROM auth.users
WHERE created_at >= NOW() - INTERVAL '12 weeks'
GROUP BY 1
ORDER BY 1 DESC;
```

Total User Count

```
SELECT COUNT(*) AS total_users FROM auth.users;
```

Users Who Signed Up But Never Got the Free Role (Trigger Failure Check)

```
-- Should return 0 rows if everything is working correctly
SELECT u.id, u.email, u.created_at
FROM auth.users u
LEFT JOIN public.user_roles ur ON ur.user_id = u.id AND ur.role = 'free'
WHERE ur.user_id IS NULL;
```

If this returns rows, run the repair SQL for each affected user:

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('PASTE-USER-UUID-HERE', 'free')
ON CONFLICT (user_id, role) DO NOTHING;

INSERT INTO public.profiles (id, email)
SELECT id, email FROM auth.users WHERE id = 'PASTE-USER-UUID-HERE'
ON CONFLICT (id) DO NOTHING;
```

Users With No Profile (Trigger Failure Check)

```
SELECT u.id, u.email, u.created_at
FROM auth.users u
LEFT JOIN public.profiles p ON p.id = u.id
WHERE p.id IS NULL;
```

All Admins

```
SELECT p.email, p.full_name, ur.granted_at
FROM public.user_roles ur
JOIN public.profiles p ON p.id = ur.user_id
WHERE ur.role = 'admin'
ORDER BY ur.granted_at;
```

Subscription Status Breakdown

```
SELECT
  status,
  plan,
  COUNT(*) AS count
FROM public.subscriptions
GROUP BY status, plan
ORDER BY status, plan;
```

Recently Active Subscribers (Signed Up in Last 90 Days)

```
SELECT
  p.email,
  p.full_name,
  p.org_name,
  p.created_at AS joined,
  s.plan,
  s.status
FROM public.profiles p
JOIN public.subscriptions s ON s.user_id = p.id
JOIN public.user_roles ur ON ur.user_id = p.id AND ur.role = 'subscriber'
WHERE p.created_at >= NOW() - INTERVAL '90 days'
ORDER BY p.created_at DESC;
```

Quick Reference: Role Grant SQL Snippets

For fast copy-paste during support requests. In every snippet, replace **PASTE-USER-UUID-HERE** with the UUID from:

```
SELECT id FROM auth.users WHERE email = 'user@example.com';
```

Grant subscriber:

```
INSERT INTO public.user_roles (user_id, role) VALUES ('PASTE-USER-UUID-HERE',
'subscriber') ON CONFLICT (user_id, role) DO NOTHING;
```

Grant advisory:

```
INSERT INTO public.user_roles (user_id, role) VALUES ('PASTE-USER-UUID-HERE',
'advisory') ON CONFLICT (user_id, role) DO NOTHING;
```

Grant admin:

```
INSERT INTO public.user_roles (user_id, role) VALUES ('PASTE-USER-UUID-HERE',
'admin') ON CONFLICT (user_id, role) DO NOTHING;
```

Revoke subscriber:

```
DELETE FROM public.user_roles WHERE user_id = 'PASTE-USER-UUID-HERE' AND role = 'subscriber';
```

Revoke advisory:

```
DELETE FROM public.user_roles WHERE user_id = 'PASTE-USER-UUID-HERE' AND role = 'advisory';
```

See all roles for a user:

```
SELECT role, granted_at, expires_at FROM public.user_roles WHERE user_id = 'PASTE-USER-UUID-HERE';
```